

位图图形工具开发

Contents

1	位图图形工具开发	1
1.1	项目概述	1
1.2	交互式工具模式 (CLI Workflow)	1
1.2.1	通用反馈与错误处理	2
1.3	实现要求	2
1.4	任务一：位图加载与分析	2
1.5	任务二：基础图形绘制	3
1.6	任务三：几何变换	4
1.7	任务四：矩形检测	5
1.8	任务五：连通组件分析	6
1.9	任务六：多边形渲染	7
1.9.1	基于像素中心法的扫描线算法原理	7
1.10	任务七：拓扑特征分析	8
1.11	提交要求和考核标准	9
1.11.1	输入输出样例与运行时长限制	9
1.11.2	编码要求	9
1.11.3	评分标准	9
1.11.4	提交要求	10

1 位图图形工具开发

1.1 项目概述

本项目旨在引导你从零开始，通过一系列递进的编程任务，构建一个功能完备的位图图形处理工具集。你将实现从基础的像素操作、图形绘制，到复杂的图像分析与算法应用。这个过程不仅会巩固你的编程基础，还将全面锻炼你在图形学、图像处理和算法设计方面的综合能力。

1.2 交互式工具模式 (CLI Workflow)

你的程序最终应以一个交互式命令行工具的形式运行。程序启动后，会进入一个命令循环，等待用户输入指令。所有操作都围绕一个当前在内存中的“活动位图”展开。“活动位图”也就是指目前正在被操作的位图，在程序运行过程中，可以有多个位图，但是只有一个位图是活动的。

- 启动与状态：程序启动后，不存在活动的位图。用户必须先使用 `new` 或 `load` 创建一个。
- 提示符：程序应显示一个提示符`>`，来等待用户输入。

1.2.1 通用反馈与错误处理

为了保证工具行为的一致性，所有命令都应遵循以下反馈和错误处理准则。

- 成功反馈：除 `render` 和 `exit` 外，所有成功执行修改、加载或分析的命令，都应输出一行简洁的成功信息（具体格式见各任务示例），以确认操作已完成。若命令未在示例中明确成功信息，请使用合理的描述，如 `Bitmap rotated.`。
- 错误处理：当操作失败时，应输出一行以 `Error:` 开头的错误信息，并且不应导致程序终止。
 - 参数错误：当命令接收到错误的参数数量、类型或值（如负数尺寸、非数字坐标）时，应报告类似 `Error: Invalid arguments.` 的错误。
 - 状态错误：在没有活动位图时调用需要位图的命令（如 `info`, `save`, `draw_*` 等），应报告 `Error: No active bitmap.`。
 - 文件错误：当无法读取或写入文件（如文件不存在、格式错误、无权限）时，应报告具体原因，如 `Error: Cannot open file '...'` 或 `Error: Invalid file format.`。

1.3 实现要求

我们不仅会运行测试数据来评判你的得分，也会阅读你提交的代码来评判你的得分。因此，所有命令的实现函数名需固定，命名规则如下：

- `new` 命令对应函数名为 `do_new`
- `load` 命令对应函数名为 `do_load`
- `save` 命令对应函数名为 `do_save`
- `render` 命令对应函数名为 `do_render`
- `exit` 命令对应函数名为 `do_exit`
- 其他命令以此类推，命令名加前缀 `do_` 作为函数名

请严格按照上述命名规范实现各命令对应的函数。此外，你应该编写合适的注释，这更有利于我们理解你的代码，从而得到更高的分数。

1.4 任务一：位图加载与分析

目标：实现位图的创建、加载，并分析其基本属性。

CLI 命令：

- `render`：打印当前活动位图的内容。
- `exit`：退出程序。
- `new <width> <height>`：创建一个用 `.` 字符填充的指定尺寸的空白位图，并将其设为活动位图（这将覆盖之前的活动位图），保证 `width` 和 `height` 都是不超过 2000 的正整数。
- `load <filepath>`：读取文件，若格式有效则加载到内存，并将其设为活动位图（这将覆盖之前的活动位图）；若文件不存在或格式错误，应报告错误。加载成功后自动显示位图的详细信息（宽度、高度、实体像素数、空白像素数、是否为空）。
- `save <filepath>`：将当前活动位图的内容保存到指定文件。

技术细节与定义:

- 坐标系: 原点 (0, 0) 位于图像的左上角, x 轴向右增长, y 轴向下增长。坐标 (x , y) 对应第 y 行, 第 x 列。
- 位图文件格式:
 - 纯文本文件 (UTF-8 编码), 由 . (背景像素) 和 # (实体像素) 构成。
 - 文件内容必须为矩形字符区域, 所有行的字符数相同。行与行之间由 `\n` 分隔。
 - 文件为空或行长度不一, 均视为格式错误。文件末尾的多余空行应被安全地忽略。

输入输出样例:

1. 输入文件 (**sample.txt**):

```
...#..  
###.##  
.....  
#####
```

2. 交互会话:

```
> load sample.txt  
Bitmap 'sample.txt' loaded.  
Width: 6  
Height: 4  
Solid Pixels: 12  
Empty Pixels: 12  
Is Empty: false  
> render  
...#..  
###.##  
.....  
#####  
> exit
```

3. 错误处理会话:

```
> load non_existent_file.txt  
Error: Cannot open file 'non_existent_file.txt'.  
> load invalid_format.txt  
Error: Invalid file format.  
> render  
Error: No active bitmap.
```

1.5 任务二: 基础图形绘制

目标: 在活动位图上实现基础的图形绘制功能。

CLI 命令:

- `draw_pixel <x> <y>`: 将位图 (x , y) 位置的字符修改为 #, 下述的填充命令都指将对应区域的字符修改为 #。

- `draw_hline <y> <x0> <x1>`: 填充第 `y` 行, `x` 坐标从 `x0` 到 `x1` (含两端) 的所有像素。
- `draw_vline <x> <y0> <y1>`: 填充第 `x` 列, `y` 坐标从 `y0` 到 `y1` (含两端) 的所有像素。
- `draw_rect <x> <y> <width> <height>`: 填充以 `(x, y)` 为左上角, 覆盖 `x` 范围从 `x` 到 `x+width-1`, `y` 范围从 `y` 到 `y+height-1` 的区域 (都含两端)。

技术细节与定义:

- 边界与剪裁 (**Clipping**): 所有绘制操作都必须进行剪裁。任何尝试在位图范围 `[0, width-1]`, `[0, height-1]` 之外的绘制都应被安全地忽略, 而不应报错。
- 参数校验: 若 `x0 > x1` 或 `y0 > y1`, 或 `width/height` 小于 1, 对应命令不执行任何操作。

输入输出样例:

1. 交互会话:

```
> new 10 5
New 10x5 bitmap created.
> draw_rect 1 1 5 3
Rectangle drawn.
> render
.....
.#####....
.#####....
.#####....
.....
> save my_rect.txt
Bitmap saved to 'my_rect.txt'.
> draw_pixel 0 0
Pixel drawn.
> draw_hline 3 5 0
> draw_hline 0 7 8
Horizontal line drawn.
> draw_vline 9 1 9
Vertical line drawn.
> render
#.....##.
.#####...#
.#####...#
.#####...#
.#####...#
.....#
```

2. 输出文件 (`my_rect.txt`):

```
.....
.#####....
.#####....
.#####....
.....
```

1.6 任务三: 几何变换

目标: 对活动位图进行几何变换。

CLI 命令:

- `flip_h`: 对活动位图进行水平翻转。
- `flip_v`: 对活动位图进行垂直翻转。
- `rotate_90`: 对活动位图进行顺时针 90 度旋转。

技术细节与定义:

- 核心逻辑: 为了模拟真实工具的行为, 所有变换操作都直接修改当前内存中的活动位图。
- `flip_h`: 原 (x, y) 处的像素移动到 $(width-1-x, y)$, 执行成功后输出 `Bitmap flipped horizontally.`
- `flip_v`: 原 (x, y) 处的像素移动到 $(x, height-1-y)$, 执行成功后输出 `Bitmap flipped vertically.`
- `rotate_90`: 此操作会改变位图尺寸。原 (W, H) 的位图会变为 (H, W) 。原 (x, y) 处的像素移动到新尺寸位图的 $(y, W-1-x)$, 执行成功后输出 `Bitmap rotated.`

输入输出样例:

1. 输入文件 (`shape_L.txt`):

```
##..  
#...  
#...  
#...
```

2. 交互会话:

```
> load shape_L.txt  
Bitmap 'shape_L.txt' loaded.  
Width: 4  
Height: 4  
Solid Pixels: 5  
Empty Pixels: 11  
Is Empty: false  
> rotate_90  
Bitmap rotated.  
> render  
....  
....  
#...  
####
```

1.7 任务四: 矩形检测

目标: 检测目前活动位图中的“面积最大”实心矩形。

CLI 命令:

- `detect_rects`: 执行检测并打印结果。

技术细节与定义:

- **最大矩形**: 一个由 # 构成的面积最大的实心矩形。如果有多个面积相同的最大实心矩形, 则输出左上角顶点坐标最小的 (先按 y 坐标从小到大, 如果 y 坐标相同则按 x 坐标从小到大)。

输入输出样例:

1. 输入文件 (**rects.txt**):

```
.##.##.  
.##.##.  
.....  
.####.  
.####.
```

2. 交互会话:

```
> load rects.txt  
Bitmap 'rects.txt' loaded.  
Width: 7  
Height: 5  
Solid Pixels: 18  
Empty Pixels: 17  
Is Empty: false  
> detect_rects  
Largest rectangle: x=1, y=3, width=5, height=2
```

1.8 任务五: 连通组件分析

目标: 识别并分析目前活动位图中的独立图形。

CLI 命令:

- **analyze_components**: 执行分析并打印结果。

技术细节与定义:

- **4-连通**: 一个实体像素 (x, y) 仅与其上、下、左、右四个方向的邻居实体像素被视为直接相连。
- **边界框 (Bounding Box)**: 边界框为组件的最小外接矩形, 即包含所有组件像素的最小矩形。你需要计算出组件的边界框, 并输出边界框的左上角坐标和宽高。
- **组件 ID**: 分析时应为每个组件分配一个从 1 开始的唯一 ID。组件按照其边界框左上角坐标的顺序排序 (先按 y 坐标从小到大, 如果 y 坐标相同则按 x 坐标从小到大), 并按此顺序分配 ID 以确保输出一致。

输入输出样例:

1. 输入文件 (**components.txt**):

```
#.#.#  
###.#
```

```
#.#..#  
.....  
.####.
```

2. 交互会话:

```
> load components.txt  
Bitmap 'components.txt' loaded.  
...(Bitmap信息)  
> analyze_components  
Total components: 3  
---  
Component 1:  
  Area: 5  
  Bounding Box: x=0, y=0, width=3, height=3  
---  
Component 2:  
  Area: 3  
  Bounding Box: x=5, y=0, width=1, height=3  
---  
Component 3:  
  Area: 4  
  Bounding Box: x=1, y=4, width=4, height=1
```

1.9 任务六：多边形渲染

目标：扩展工具集以支持更复杂的图形渲染，绘制多边形。

CLI 命令:

- `draw_poly <x1> <y1> <x2> <y2> ... <xN> <yN>`

技术细节与定义:

- 多边形参数校验: `draw_poly` 命令要求至少提供 3 个顶点（即 6 个坐标参数），否则不执行任何操作并报告参数错误。同时，假定输入顶点构成简单多边形（边不自交），顶点按照绘制顺序生成，两个相邻顶点之间用直线连接。
- 像素中心法: 为了确保结果的唯一性，我们采用像素中心法 (Pixel Center Rule) 来填充像素。一个像素 (x, y) 被填充当且仅当其中心点 (x, y) 位于多边形的内部或多边形的边界。其判断方法是：从像素中心点 (x, y) 向 X 轴正方向发射一条射线。如果该射线与多边形边界的交点数为奇数，则像素被填充。多边形的边界上的像素需要额外判断。

1.9.1 基于像素中心法的扫描线算法原理

扫描线算法是基于像素中心法的多边形填充方法。核心思想是：逐行扫描位图，对每一行计算该行与多边形边界的交点，然后根据像素中心法规则填充内部区域。

算法流程:

1. 预处理：将多边形的所有边整理成边表，记录每条边的起止 y 坐标、起点 x 坐标和斜率信息。跳过水平边（单独处理）。
2. 确定扫描范围：找到多边形所有顶点的最小和最大 y 坐标，作为扫描线的范围。
3. 逐行扫描：
 - 对每条扫描线 y，计算它与所有边的交点 x 坐标，取最小值和最大值作为下一步扫描的起点和终点
 - 对于起点和终点之间的所有点，以像素中心法判断该点是否在多边形内部，若在内部则填充该像素，即从像素点向 X 轴正方向发射一条射线，若与多边形边界的交点数为奇数，则像素被填充。

Tips:

- 边界处理：为避免顶点重复计算，通常采用“左闭右开”原则，即边的上端点不参与是否为像素中心法中射线与多边形边界的交点的计算。
- 水平边：水平边不参与射线与多边形边界的交点的计算，需要单独绘制
- 浮点精度：本题计算可能会涉及浮点运算，转换为整数坐标时需要适当的舍入处理。对于有浮点数参与的计算，建议使用微小量`eps`来避免浮点误差导致的结果不准确。例如计算判断两个点是否在同一点时，可以使用`abs(x1-x2) < eps 且 abs(y1-y2) < eps`来判断。我们建议使用`1e-6`作为`eps`的值。

输入输出样例:

1. 交互会话:

```
> new 12 6
New 12x6 bitmap created.
> draw_poly 2 1 10 1 6 5
Polygon drawn.
> render

.....
..#####.
...#####.
....#####.
.....#####.
.....###.
.....#.....
.....#.....
```

1.10 任务七：拓扑特征分析

目标：分析复杂图形的拓扑特征。

CLI 命令:

- `analyze_topo`

技术细节与定义:

- 孔洞：一个由 `.` 构成的、有限的 **4-连通** 区域，且该区域被 `#` 完全包围。图像最外层的无垠背景不计为孔洞。

- 连通分量：一个由 # 构成的连通区域，即从任意一个 # 像素出发，可以到达其他任意一个 # 像素。
- 分支点：一个 # 像素，如果移除它会导致全图中 # 组成的连通分量数量增加。

输入输出样例：

1. 输入文件 (**topo.txt**):

```
.###.  
.#.#.  
.###.  
..#..  
.###.
```

2. 交互会话:

```
> load topo.txt  
Bitmap 'topo.txt' loaded.  
...  
> analyze_topo  
Holes found: 1  
Branch points found: 3  
Branch points coordinates: (2, 2) (2, 3) (2, 4)
```

1.11 提交要求和考核标准

1.11.1 输入输出样例与运行时长限制

为了方便大家进行调试，我们在 `testdata` 目录中给出了样例。

给出的测试样例包括三个文本文件：

- `bitmap.txt` 为需要读入的位图
- `commands.txt` 为输入命令
- `results.txt` 为正确的标准输出的内容

在测试过程中，每个测试占用的内存不超过 `4 GiB`，时间运行不超过 `10` 秒钟。

1.11.2 编码要求

语言不限，但根据题目选择合适的语言并按照要求进行设计和编码。

1.11.3 评分标准

设计和实现程序，完成前述功能。同时，请注意代码规范、命名标准、注释和说明。如果不能完成全部程序功能，也请不要担心，我们会对每个任务独立评分。评分过程会检查代码逻辑，因此即使输出结果错误，也可以得到一定分数。

分项	分数
任务一：位图加载与分析	15
任务二：基础图形绘制	15
任务三：几何变换	10
任务四：矩形检测	10
任务五：连通组件分析	10
任务六：多边形渲染	10
任务七：拓扑特征分析	10
小型综合测试	5
大型综合测试	5
代码风格	10

1.11.4 提交要求

请将程序源代码使用 7z 格式压缩后命名为“姓名.7z”并等待监考老师收取。

压缩包中应仅包含源代码，不包含测试用的输入输出、中间文件或可执行文件。压缩包大小原则上不超过 5MiB。